

Experiment No:1

Experiment Name: 8 to 3 Encoder

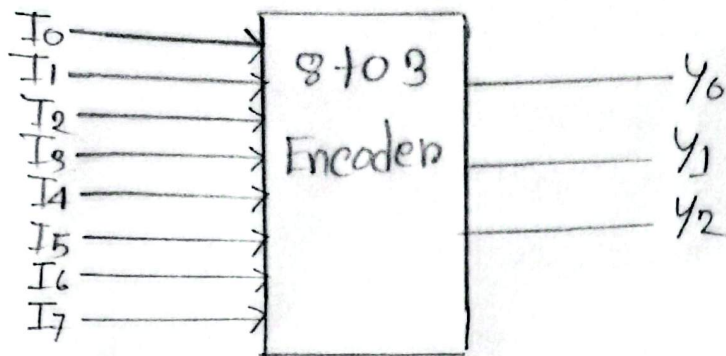
Objective: To design and implement an 8 to 3 priority encoder using logic gates and verify its operation using a truth table.

Theory: An encoder is a combinational logic circuit that converts 2^n input line into n output line. An 8 to 3 encoder takes 8 inputs (I_0 - I_7) and produces a 3 bit binary output (Y_0, Y_1, Y_2). at any given time, only one input line is active (High), and the corresponding binary code appears at the output. It is the reverse operation of a decoder.

Requirement Components:

1. OR gates (7432)
2. ~~NOT gates (7404)~~
2. LEDs
3. Connecting wires

Block diagram:



Truth table:

Input								Output		
I ₇	I ₆	I ₅	I ₄	I ₃	I ₂	I ₁	I ₀	Y ₀	Y ₁	Y ₂
0	0	0	0	0	0	0	1	0	0	0
0	0	0	0	0	0	1	0	0	0	1
0	0	0	0	0	1	0	0	0	1	0
0	0	0	0	1	0	0	0	0	1	1
0	0	0	1	0	0	0	0	1	0	0
0	0	1	0	0	0	0	0	1	0	1
0	1	0	0	0	0	0	0	1	1	0
1	0	0	0	0	0	0	0	1	1	1

Function:

$$Y_0 = I_4 + I_5 + I_6 + I_7$$

$$Y_1 = I_2 + I_3 + I_6 + I_7$$

$$Y_2 = I_1 + I_3 + I_5 + I_7$$

Diagram:

Result and Conclusion:

The 8-to-3 encoder circuit was successfully implemented and tested. For each active input line (I_0 - I_7), the correct 3 bit binary code was obtained at the output, verifying the truth table. The circuit operates correctly as an encoder.

Experiment No: 02

Experiment Name: 3 to 8 Decoder

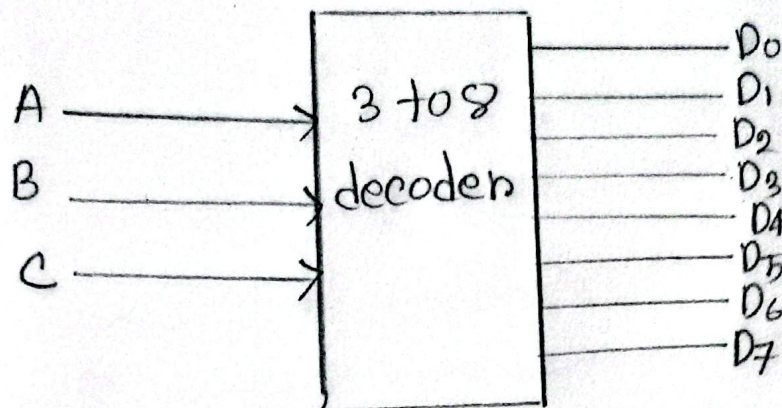
Objective: To design and implement a 3 to 8 line decoder using logic gates and verify its operation using a truth table.

Theory: A decoder is a combinational circuit that converts n input lines into 2^n output lines. A 3 to 8 decoder accepts a 3 bit binary input (A, B, C) and activates one of 8 output lines ($D_0 - D_7$) corresponding to the input combination. Only one output is HIGH at a time. Decoders are widely used in memory address decoding and data demultiplexing.

Requirement Components:

1. IC 74138 (3 to 8 Decoder) or AND gates.
2. NOT gates
3. LEDs
4. Connecting wires

Block diagram:



Truth table:

Input			Output							
A	B	C	D ₀	D ₁	D ₂	D ₃	D ₄	D ₅	D ₆	D ₇
0	0	0	1	0	0	0	0	0	0	0
0	0	1	0	1	0	0	0	0	0	0
0	1	0	0	0	1	0	0	0	0	0
0	1	1	0	0	0	1	0	0	0	0
1	0	0	0	0	0	0	1	0	0	0
1	0	1	0	0	0	0	0	1	0	0
1	1	0	0	0	0	0	0	0	1	0
1	1	1	0	0	0	0	0	0	0	1

Function:

$$D_0 = \bar{A}\bar{B}\bar{C}$$

$$D_1 = \bar{A}\bar{B}C$$

$$D_2 = \bar{A}B\bar{C}$$

$$D_3 = \bar{A}BC$$

$$D_4 = A\bar{B}\bar{C}$$

$$D_5 = A\bar{B}C$$

$$D_6 = AB\bar{C}$$

$$D_7 = ABC$$

Diagram:

Result and Conclusion:

The 3-to-8 decoder was successfully implemented. For each 3-bit binary input combination, exactly one of the eight output lines become HIGH, confirming the correct operation. The circuit matched the theoretical truth table perfectly.

Experiment No: 03

Experiment Name: 8:1 multiplexer

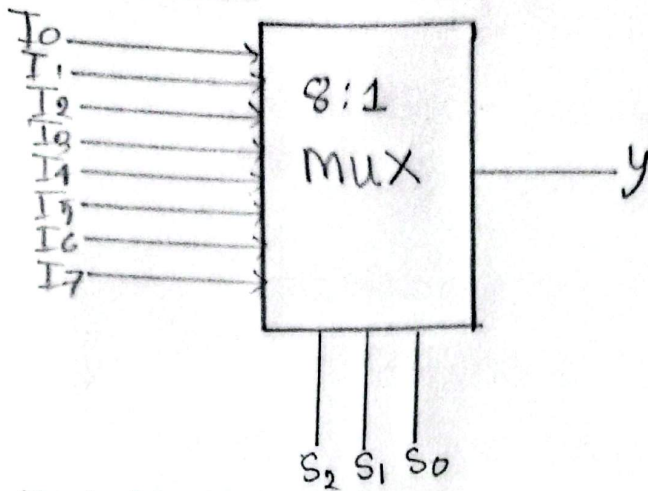
Objective: To design and implement an 8:1 multiplexer using logic gates and verify its operation.

Theory: A multiplexer (mux) is a combinational circuit that selects one of many input data lines and routes it to a single output. An 8:1 mux has 8 data inputs (I_0 - I_7), 3 select lines (S_2, S_1, S_0) and 1 output (Y). Based on the 3 bit binary combination of the select lines, one of the 8 data inputs is passed to the output. It is also known as a data selector.

Requirement Components:

1. IC 74151 (8:1 mux) or AND/OR/NOT gates
2. Power Supply
3. LEDs.
4. Connecting Wires

Block diagram:



Truth table:

S_2	S_1	S_0	y
0	0	0	I_0
0	0	1	I_1
0	1	0	I_2
0	1	1	I_3
1	0	0	I_4
1	0	1	I_5
1	1	0	I_6
1	1	1	I_7

Function:

$$y = \bar{S}_2 \cdot \bar{S}_1 \cdot \bar{S}_0 \cdot I_0 + \bar{S}_2 \cdot \bar{S}_1 \cdot S_0 \cdot I_1 + \bar{S}_2 \cdot S_1 \cdot \bar{S}_0 \cdot I_2 + \bar{S}_2 \cdot S_1 \cdot S_0 \cdot I_3 \\ + S_2 \cdot \bar{S}_1 \cdot \bar{S}_0 \cdot I_4 + S_2 \cdot \bar{S}_1 \cdot S_0 \cdot I_5 + S_2 \cdot S_1 \cdot \bar{S}_0 \cdot I_6 + S_2 \cdot S_1 \cdot S_0 \cdot I_7$$

Diagram:

Result and Conclusion:

The 8:1 multiplexer circuit was built and verified. For each combination of select lines, the corresponding data input was correctly routed to the input, confirming the operation of the multiplexer.

Experiment No: 04

Experiment Name: 1:8 Demultiplexer

Objective: To design and implement a 1:8 demultiplexer using logic gates and verify its operation.

Theory: A demultiplexer (DEMUX) performs the reverse operation of a multiplexer. It takes a single input data line and routes it to one of 2^n output lines based on n select lines. A 1:8 DEMUX has 1 input (I), 3 select lines (S_2, S_1, S_0) and 8 outputs ($D_0 - D_7$). The input signal is forwarded to only one output at a time, determined by the select line combination.

Requirement Components:

1. IC 74138 (used as DEMUX) or AND/NOT gates

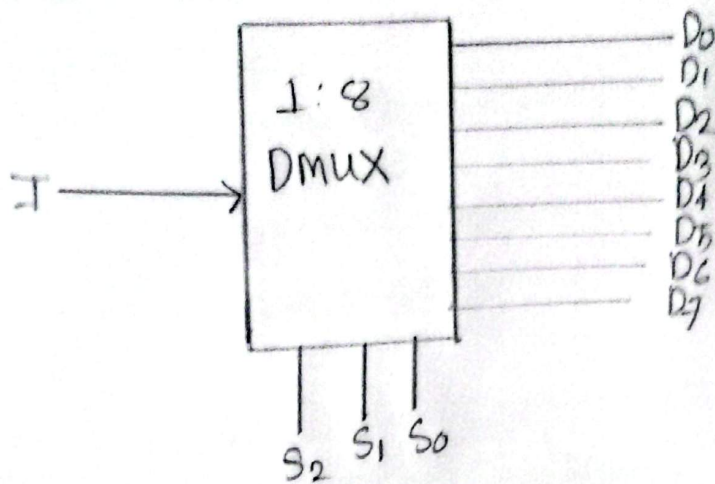
2. Breadboard

3. Power Supply.

4. LEDs.

5. Connecting wires.

Block diagram:



Truth table:

I	S ₂	S ₁	S ₀	D ₇	D ₆	D ₅	D ₄	D ₃	D ₂	D ₁	D ₀
1	0	0	0	0	0	0	0	0	0	0	I
1	0	0	1	0	0	0	0	0	0	I	0
1	0	1	0	0	0	0	0	0	I	0	0
1	0	1	1	0	0	0	0	I	0	0	0
1	1	0	0	0	0	0	I	0	0	0	0
1	1	0	1	0	0	I	0	0	0	0	0
1	1	1	0	0	I	0	0	0	0	0	0
1	1	1	1	I	0	0	0	0	0	0	0

Function:

$$D_0 = \bar{S}_2 \cdot \bar{S}_1 \cdot \bar{S}_0 \cdot I$$

$$D_2 = \bar{S}_2 \cdot \bar{S}_1 \cdot S_0 \cdot I$$

$$D_4 = \bar{S}_2 \cdot S_1 \cdot \bar{S}_0 \cdot I$$

$$D_6 = \bar{S}_2 \cdot S_1 \cdot S_0 \cdot I$$

$$D_1 = S_2 \cdot \bar{S}_1 \cdot \bar{S}_0 \cdot I$$

$$D_3 = S_2 \cdot \bar{S}_1 \cdot S_0 \cdot I$$

$$D_5 = S_2 \cdot S_1 \cdot \bar{S}_0 \cdot I$$

$$D_7 = S_2 \cdot S_1 \cdot S_0 \cdot I$$

Diagram:

Result and conclusion:

The 1:8 demultiplexer was successfully implemented and tested. The single input data was correctly routed to the selected output line for all 8 combinations of select lines, confirming correct DEMUX operation.

Experiment No: 05

Experiment Name: Implement the Given Function using 8:1 multiplexer.

Objective: To implement the Boolean function $F(A, B, C, D) = \sum(4, 5, 7, 9, 11, 12, 13, 15)$ using an 8:1 MUX.

Theory: A multiplexer can be used to implement any Boolean function. For a function of 4 variables using an 8:1 MUX, one variable (A) is used as the data input, while the remaining 3. Variable (B, C, D) are connected to the select lines. Each data input of the MUX is set to 0, 1, or A (or A') based on the minterm grouping. This technique reduces hardware complexity significantly.

Requirement Components:

1. IC (74151) (8:1 MUX) or AND and OR and NOT gates.
2. LEDs.
3. Logic switches.
4. Power Supply
5. Connecting wires.

Step 1:

Truth table:

A	B	C	D	Y
0	0	0	0	0
0	0	0	1	0
0	0	1	0	0
0	0	1	1	0
0	1	0	0	1
0	1	0	1	1
0	1	1	0	0
0	1	1	1	1
1	0	0	0	0
1	0	0	1	1
1	0	1	0	0
1	0	1	1	1
1	1	0	0	1
1	1	0	1	1
1	1	1	0	0
1	1	1	1	1

Step 2:

input line A, Selection line B, C, D

Step 3:

	A ₀	A ₁	A ₂	A ₃	A ₄	A ₅	A ₆	A ₇
$\bar{A}$	0	1	2	3	④	⑤	6	⑦
A	8	⑨	10	⑪	⑫	⑬	14	⑮
	0	A	0	A	1	1	0	1

Diagram:

Result and Conclusion:

The given Boolean function $F(A, B, C, D) = \sum(4, 5, 7, 9, 11, 12, 13, 15)$ was successfully implemented using the 8:1 mux. The output matched the function values for all 16 input combinations, confirming correct implementation.

Experiment No: 06

Experiment name: Design a S-R Flip-Flop

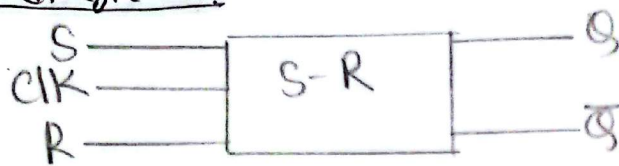
Objective: To design and implement an SR (Set-Reset) Flip-flop using NAND/NOR gates and verify its characteristic table.

Theory: An S-R flip-flop is a basic sequential logic circuit with two inputs: S and R and two outputs: Q and Q'. When $S=1, R=0$ the output Q is set to 1, when $S=0, R=1$, the output Q is reset to 0. When $S=0, R=0$, the previous state is maintained (no change). When $S=1, R=1$, the output is undefined. It can be built using NAND or NOR gates. With a clock input, it becomes a clocked S-R flip-flop.

Requirement Components:

1. IC 7400 (NAND gates) or IC 7402 (NOR gates)
2. LEDs.
3. Push buttons.
4. Connecting wires.

Block diagram:



Truth table:

S	R	Q _{n+1}
0	0	Nochange
0	1	0
1	0	1
1	1	Invalid

Diagram:

Result and conclusion:

The S-R flip-flop was successfully designed and verified. The output Q correctly changed to 1 on set, to 0 on reset, held previous state on no-change input, and confirmed the forbidden state condition matching the characteristic table.

Experiment No: 07

Experiment Name: Design a J-K flip-Flop

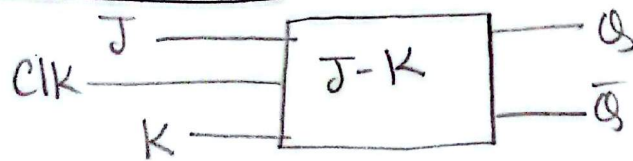
Objective: To design and implement a J-K flip-flop and verify its characteristic table.

Theory: The J-K flip-flop is an improved version of the S-R flip-flop that eliminates the forbidden state. It has two inputs J and K. When $J=0, K=0$ the output holds, when $J=0, K=1$ output resets to 0, when $J=1, K=0$ output sets to 1. When $J=1, K=1$, the output toggles. This makes the J-K flip-flop the most versatile flip-flop. It can be constructed from an S-R flip-flop by adding feedback.

Requirement Components:

1. IC 7476 (J-K Flip-Flop) or NAND gates.
2. Power Supply
3. Clock generator
4. LEDs.
5. Connecting wires.

Block diagram:



Truth table:

J	K	Q_{n+1}
0	0	Hold
0	1	0
1	0	1
1	1	Toggle

Diagram:

Result and conclusion:

The J-K flip-flop was successfully implemented. The output correctly held, reset, set and toggled for the four input combinations, confirming the characteristic table. The toggling feature was clearly observed when $J=K=1$.

Experiment No:08

Experiment Name: Design a T-Flip-Flop

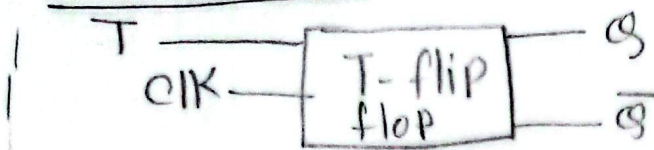
Objective: To design and implement a T flip-flop and verify its characteristic table.

Theory: A T flip-flop has a single input T. It is derived from the J-K flip-flop by connecting both J and K inputs together ($J=K=T$). When $T=0$, the output does not change (holds). When $T=1$, the output toggles. T flip-flops are widely used in counters and frequency dividers because of their toggling property. A single T flip-flop divides the input clock frequency by 2.

Requirement Components:

1. IC 7476 (JK as T Flip-flop)
2. Power supply.
3. clock generator
4. LEDs.
5. Connecting wires.

Block diagram:



Truth table:

T	Q _{present}	Q _{next}	Operation
0	0	0	no change
0	1	1	no change
1	0	1	Toggle
1	1	0	Toggle

Function:

$$Q = T \oplus Q \text{ (XOR operation)}$$

Diagram:

Result and Conclusion:

The T-flip-flop was successfully designed using a J-K flip-flop with J and K tied together. The output correctly toggled when $T=1$ and held when $T=0$. The frequency division property was also verified.

Experiment No: 00

Experiment Name: Design a D Flip-Flop.

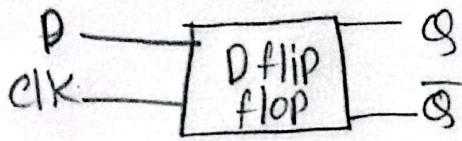
Objective: To design and implement a D flip-flop and verify its characteristic table.

Theory: A D flip-flop is the simplest flip-flop with a single data input D. On the active clock edge the output Q takes the value of D. It essentially delays the input by one clock cycle, which is why it is called a Delay flip-flop. The forbidden state of the S-R flip-flop is eliminated by tying $S=D$ and $R=D'$. D flip-flops are fundamental in data storage registers and synchronous sequential circuits.

Requirement Components:

1. IC 7474
2. Breadboard
3. Power Supply
4. Clock generator
5. LEDs.
6. Connecting wires.

Block diagram:



Truth table:

CLK	D	Q_{n+1}	Operation
1	0	0	Reset
1	1	1	set

Function:

$$Q_{n+1} = D$$

Diagram:

Result and conclusion:

The D flip-flop was successfully implemented. The output Q followed the input D on each clock pulse, confirming the data latching behavior. The circuit correctly stored the binary input transferred it to output on the clock edge.

Experiment No: 10

Experiment Name: Design an Asynchronous 3-bit Up Counter

Objective: To design and implement an asynchronous (ripple) 3 bit up counter using J-K flip-flops and verify its operation.

Theory: An asynchronous counter is a sequential circuit in which flip-flops are triggered one after another, not simultaneously. In a 3-bit up counter, three JK flip-flop are connected in cascade. The output of each flip-flop becomes the clock for the next. The counter counts from 000 to 111 (0 to 7) and then resets to 000. It is called asynchronous because not all flip-flops change at the same time - there is a propagation delay.

Components:

1. 7SE G-BED
2. Clock
3. JKFF

Diagram:

Result and Conclusion:

The asynchronous 3 bit up counter was successfully implemented. The counter incremented from 000 to 111 on successive clock pulses and reset to 000 after count 7, verifying the correct up counting sequence.

Experiment No:11

Experiment Name: Design an Asynchronous 3bit Down Counter.

Objective: To design and implement an asynchronous 3bit down counter using J-K flip-flops and verify its operation.

Theory: An asynchronous 3-bit down counter counts in decreasing order from 111(7) to 000(0). It is implemented by connecting the complement output (Q') of each flip-flop to the clock input of the next flip-flop (instead of Q as in the up counter). Each JK flip flop is in toggle mode ($J=K=1$). The counter decrements on each clock pulse. After reaching 000, it wraps around to 111 and the sequence repeats.

Requirement Components:

1. 7SEG-BED
2. clock
3. JKFF

Diagram:

Result and Conclusion:

The asynchronous 3-bit down counter was successfully implemented. The counter decremented from 111 to 000 on successive clock pulses and rolled over to 111 after reaching 0, confirming the correct down-counting operation.

Experiment No. 12

Experiment Name: Design a Ripple Counter that can count both 0-7 and 7-0

Objective: To design and implement a ripple counter capable of counting both upward (0 to 7) and downward (7 to 0) using a mode control signal.

Theory: A bidirectional ripple counter can count both up and down depending on a control signal (m). When $m=0$ (up mode), the Q output of each flip-flop clock the next counting from 0 to 7. When $m=1$ (Down mode), The Q' (complement) output clocks the next flip-flop, counting from 7 to 0. The mode selection can be implemented using AND-OR logic or a 2:1 mux to select either Q or Q' as the clock source for the next stage. This combines both up and down counter functionality in a single circuit.

Requirement components:

1. 7490 - Dec
2. clock

Truth table:

m	Q	$\bar{Q}$	y
0	0	0	0
0	0	1	0
0	1	0	1
0	1	1	1
1	0	0	0
1	0	1	1
1	1	0	0
1	1	1	1

Function:

	00	01	11	10
$\bar{m}$	0	0	1	1
m	0	1	1	0

$$y = \bar{m}Q + m\bar{Q}$$

Diagram:

Result and conclusion:

The bidirectional ripple counter was successfully designed. With ~~one~~ $m=0$, the counter correctly counted up from 0 to 7 and wrapped around. With $m=1$, it counted down from 7 to 0 and wrapped around. Both counting modes were verified experimentally.